

TRƯỜNG ĐẠI HỌC KINH TẾ QUỐC DÂN – TRƯỜNG CÔNG NGHỆ
KHOA KHOA HỌC DỮ LIỆU VÀ TRÍ TUỆ NHÂN TẠO

CẨM NANG HỌC PHẦN & PHƯƠNG PHÁP TỰ HỌC

DSAI1003: CÁC NGUYÊN LÝ LẬP TRÌNH CƠ BẢN VỚI PYTHON

Fundamental Programming Concepts in Python

Giảng viên: TS. Vũ Đức Minh
Khối lượng: 3 Tín chỉ (45h LT + 22.5h TH)
Website: <https://vudmvn.github.io/Fundamental-of-Programming-Concepts-in-Python/>

Chương trình: Khoa học dữ liệu (DSFE)
Quyết định: 975/QĐ-ĐHKTQD (30/03/2024)

Nội dung Báo cáo / Cẩm nang Môn học

Phần 1: Tổng quan Học phần (Syllabus)

- ▶ Thông tin chung & Triết lý đào tạo
- ▶ Mục tiêu môn học (COs G1 – G7)
- ▶ Giáo trình chuẩn quốc tế & Đánh giá
- ▶ Lộ trình đào tạo chi tiết 15 tuần

Phần 2: Phương pháp Tự học Hiệu quả

- ▶ 4 sai lầm kinh điển & Nguyên tắc cốt lõi
- ▶ Chu trình tự học chủ động 9 bước
- ▶ Predict → Run → Explain & Teach-Back
- ▶ Debugging 8 bước & Bảng Error Log

Phần 3: Sử dụng AI Hỗ trợ Đột phá

- ▶ Định vị: AI Personal Tutor & Coach
- ▶ Khung Prompt chuẩn mực 4 thành phần
- ▶ Nguyên tắc 5T & Hệ gợi ý 3 cấp độ
- ▶ Bộ công cụ Prompt Toolkit & Thảm định

Phần 4: Kế hoạch Hành động Tuần 1

- ▶ Cài đặt môi trường (VS Code, Colab)
- ▶ Khám phá tài nguyên trên GitHub
- ▶ Khởi tạo Nhật ký học tập (Learning Log)
- ▶ Cam kết văn hóa học tập chủ động

PHẦN 1

TỔNG QUAN HỌC PHẦN (SYLLABUS)

Thông tin môn học, Mục tiêu, Giáo trình, Đánh giá & Lộ trình 15 tuần

1. Giới thiệu Học phần & Triết lý Tiếp cận

Vị trí môn học trong CTĐT

- ▶ **Mã học phần:** DSAI1003 (3 tín chỉ).
- ▶ **Đối tượng:** Sinh viên Cử nhân *Khoa học dữ liệu trong Tài chính & TMĐT (DSFE)*.
- ▶ **Tính chất:** Học phần **Bắt buộc cơ sở ngành** tiên quyết, đặt nền tảng tư duy thuật toán.

Triết lý tiếp cận môn học

- ▶ **Không đòi hỏi kinh nghiệm trước:** Chỉ cần nắm chắc đại số và tư duy logic.
- ▶ **Gắn liền ứng dụng thực tế:** Case study kinh tế, tài chính, TMĐT.
- ▶ **Từ bản chất đến nâng cao:** Thiết kế giải thuật → Cú pháp → OOP.

Cam kết chuẩn đầu ra: Đủ năng lực tự thiết kế và cài đặt chương trình hoàn chỉnh giải quyết các bài toán phân tích kinh doanh thực tế, sẵn sàng cho các môn học chuyên sâu về Máy học và Trí tuệ nhân tạo.

2. Mục tiêu Học phần (Course Learning Outcomes)

Tư duy & Kỹ năng Lập trình Cơ bản

- ▶ **G1 (Tư duy thuật toán):** Biểu diễn giải thuật bằng mã giả (pseudocode) và lưu đồ (flowchart).
- ▶ **G2 (Nền tảng Python):** Làm chủ kiểu dữ liệu, biểu thức, cấu trúc rẽ nhánh, vòng lặp và hàm.
- ▶ **G3 (Cấu trúc dữ liệu):** Quản lý dữ liệu hiệu quả với List, Tuple, Set, Dictionary và chuỗi.

Kỹ năng Thực chiến & Ứng dụng DSFE

- ▶ **G4 (Tập & Ngoại lệ):** Xử lý tập (I/O) và ngoại lệ an toàn, tin cậy.
- ▶ **G5 (Lập trình Hướng đối tượng):** Mô hình hóa thực thể bài toán qua Class, Object, Tính kế thừa.
- ▶ **G6 (Công cụ hiện đại):** Sử dụng thành thạo VS Code, Git, Google Colab và AI hỗ trợ.
- ▶ **G7 (Ứng dụng thực tế):** Giải quyết bài toán phân tích dữ liệu kinh tế – tài chính.

3. Giáo trình Chuẩn Quốc tế & Phương thức Đánh giá

3 Bộ Giáo trình Chuẩn Quốc tế

1. **[1] Python for Everyone** (3rd Ed., 2019)
Cay Horstmann, Rance Necaise (Wiley)
→ **Giáo trình chính** về cú pháp & giải thuật.
2. **[2] Intro to Python for CS and Data Science** (2020)
Paul Deitel, Harvey Deitel (Pearson)
→ **Giáo trình chính** về dữ liệu và AI.
3. **[3] The Python Workbook** (2nd Ed., 2019)
Ben Stephenson (Springer)
→ Sách bài tập tự luyện kèm lời giải.

Cơ cấu Đánh giá Kết quả (3 Đầu điểm)

Hình thức	Nội dung & Công cụ	Tỷ lệ
Bài tập về nhà	Lý thuyết & code (LMS, 2 tuần/bài)	30%
Thi giữa kỳ	Bài thi Tuần 7 (Bài 1 – 6, giấy/máy)	30%
Thi cuối kỳ	Thực hành máy phòng Lab (Tuần 15)	40%

Quy định Học phần

- ▶ **Chuyên cần:** Tham dự tối thiểu **80%** số buổi học.
- ▶ **Điều kiện đạt môn:** Điểm tổng kết $\geq 5.0 / 10.0$.

4. Kế hoạch Giảng dạy 15 Tuần – Giai đoạn 1 (Tuần 1 – 7)

Tuần	Bài học & Nội dung	Trọng tâm Kiến thức	Đánh giá
1	Bài 1: Giới thiệu & Thuật toán	Máy tính, thông dịch Python, mã giả, lưu đồ	Setup môi trường
2	Bài 2: Kiểu dữ liệu & Biểu thức	Số, chuỗi, lệnh gán, biến, toán tử, module	Bài tập #1
3	Bài 3: Cấu trúc ra quyết định	Rẽ nhánh if, if-else, if-elif-else, Boolean	Nộp BT #1
4	Bài 4: Cấu trúc vòng lặp	Vòng lặp while, for, lồng lồng nhau, chuỗi	Bài tập #2
5	Bài 5: Thiết kế hàm (Function)	Cú pháp def, tham số, return, phạm vi biến	Nộp BT #2
6	Bài 6: Cấu trúc List & Tuple	Mảng 1D, 2D, slicing, các hàm tích hợp	Bài tập #3
7	Thi giữa kỳ (Midterm Exam)	Ôn tập củng cố & thi đánh giá năng lực	Thi GK (30%)

5. Kế hoạch Giảng dạy 15 Tuần – Giai đoạn 2 (Tuần 8 – 15)

Tuần	Bài học & Nội dung	Trọng tâm Kiến thức	Đánh giá
8–9	Bài 7: Thao tác Tập & Ngoại lệ	Đọc/ghi file text, <code>sys.argv</code> , <code>try-except</code>	Bài tập #4 Case study TMĐT Mini-lab OOP
10	Bài 8: Tập hợp (Set) & Từ điển	Key-value, bảng băm, cấu trúc JSON lồng nhau	
11–12	Bài 9: Đối tượng & Lớp (OOP 1)	Tư duy hướng đối tượng, <code>class</code> , <code>__init__</code> , <code>method</code>	
13–14	Bài 10: Kế thừa & Đa hình (OOP 2)	Cây kế thừa, <code>super()</code> , ghi đè method, đa hình	Đề cương CK Mock test phòng máy
15	Tổng kết & Hướng dẫn thi	Hệ thống hóa toàn bộ kiến thức môn học	

PHẦN 2

PHƯƠNG PHÁP TỰ HỌC LẬP TRÌNH HIỆU QUẢ

Tránh bẫy ảo tưởng năng lực, Chu trình 9 bước, Học tập chủ động & Kỹ năng Debugging

6. Bản chất Học Lập trình & 4 Ngộ nhận Tai hại

"Học lập trình bằng cách đọc ít hơn, thực hành nhiều hơn và tự suy nghĩ trước khi xem lời giải."

Cái bẫy 1: Đọc lướt không chạy code

Chỉ nhìn mắt thấy code mẫu và tưởng mình đã hiểu. Khi tắt máy đi, hoàn toàn không gõ lại được một dòng.

Cái bẫy 2: Copy-paste mù quáng

Copy code trên mạng hoặc AI, code chạy được nhưng không thể giải thích tại sao câu lệnh đó tồn tại.

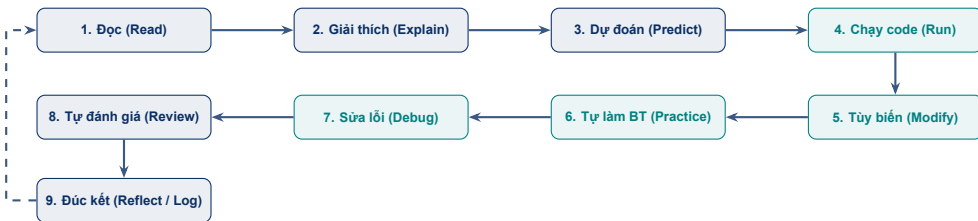
Cái bẫy 3: Xem đáp án quá sớm

Vừa gặp bài khó là mở giải ngay. Bỏ qua bước "vật lộn" rèn luyện tư duy giải quyết vấn đề của não bộ.

Cái bẫy 4: Học dồn dập nhiều chủ đề

Cố học từ biến đến OOP trong 2 ngày mà không có thời gian làm bài tập để kiến thức lắng đọng.

7. Chu trình Tự học Lập trình Chủ động 9 Bước



Phân bổ thời gian tự học vàng (Quy tắc 70/30):

Dành tối đa **30% thời gian** cho việc đọc lý thuyết; hơn **70% thời gian** phải dành trực tiếp cho: Gõ code, quan sát lỗi, phân tích traceback và giải bài tập độc lập!

8. Kỹ thuật Học tập Chủ động Đột phá

Predict → Run → Explain

Trước khi bấm nút Run trên IDE:

- ▶ Viết ra nháp chính xác màn hình sẽ in ra gì.
- ▶ Dự đoán giá trị biến trong bộ nhớ thay đổi ra sao.
- ▶ Nếu kết quả khác dự đoán → Đây là lúc học tốt nhất!

Teach-Back (Giảng ngược lại)

Thử giải thích khái niệm cho người chưa biết gì. Nếu bạn chỉ nói *"Nó chạy thế vì nó phải thế"* → Bạn chưa hiểu bản chất!

Active Recall (Gợi nhớ chủ động)

Đừng đọc lại slide nhiều lần!

- ▶ Gấp tài liệu lại và tự trả lời:
 - ▶ `input()` trả về kiểu gì?
 - ▶ Ép kiểu số nguyên dùng hàm nào?
 - ▶ Tại sao chuỗi + số lại lỗi?

Spaced Repetition (Lặp lại ngắt quãng)

Ôn tập theo chu kỳ giãn cách:

Ngày 1 → Ngày 3 → Ngày 7 → Ngày 14

Biến kiến thức tạm thời thành phản xạ tư duy dài hạn.

9. Nấc thang Luyện tập 5 Cấp độ (Learning Ladder)

Cấp 5 – Mini Project: Xây dựng ứng dụng console quản lý đơn hàng, phân tích tệp giao dịch.

Cấp 4 – Bài toán thực tế: Tính tiền điện lũy tiến, tính thuế TNCN, tính lãi suất kép ngân hàng.

Cấp 3 – Kết hợp khái niệm: Kết hợp đồng thời vòng lặp for và cấu trúc rẽ nhánh if-else.

Cấp 2 – Viết lại từ mô tả: Che lời giải mẫu, đọc đề bài và tự mình viết code từ con số 0.

Cấp 1 – Sửa ví dụ có sẵn: Đổi giá trị biến, đổi kiểu số nguyên sang số thực, quan sát kết quả.

Không bao giờ nhảy cóc từ ví dụ cơ bản lên ngay bài toán phức tạp mà bỏ qua các nấc thang trung gian!

10. Kỹ năng Debugging & Phân loại Lỗi Tiêu chuẩn

Quy trình 8 bước Debugging tiêu chuẩn

1. Đọc toàn bộ lỗi từ dòng cuối ngược lên.
2. Định vị chính xác số dòng gây ra lỗi.
3. Nhận diện kiểu lỗi (TypeError, IndexError...).
4. Dùng print() kiểm tra giá trị biến tức thời.
5. Dùng hàm type() kiểm tra kiểu dữ liệu biến.
6. So sánh output thực tế vs. output kỳ vọng.
7. Thử nghiệm với bộ input tối giản nhất.
8. **Chỉ sửa 1 yếu tố** duy nhất trong mỗi lần thử!

Phân biệt 3 nhóm lỗi lập trình

- ▶ **Syntax Error (Lỗi cú pháp):** Viết sai ngữ pháp ngôn ngữ (quên dấu :, đóng ngoặc). Python phát hiện ngay khi nạp.
- ▶ **Runtime Error (Thời gian chạy):** Cú pháp đúng nhưng sập khi chạy (chia cho 0, ép kiểu chuỗi chữ thành số nguyên).
- ▶ **Logic Error (Lỗi ngữ nghĩa):** Code chạy êm ru không báo lỗi nhưng **kết quả tính sai!** (Nguy hiểm nhất).

11. Nhật ký Học tập & Sổ tay Bắt lỗi (Error Log)

Biến lỗi sai thành tài sản học tập cá nhân:

Mỗi lỗi sai gặp phải khi được ghi lại cẩn thận sẽ trở thành kinh nghiệm quý báu giúp bạn không bao giờ mắc lại trong phòng thi.

Lỗi gặp phải	Nguyên nhân gốc rễ	Cách khắc phục	Hiểu?
TypeError: can only concatenate str to str	input() luôn trả về chuỗi kí tự, không thể trực tiếp số.	Ép kiểu: <code>int(input())</code> hoặc <code>float(input())</code> .	✓
IndexError: list index out of range	Chỉ số mảng từ 0 đến $N - 1$, duyệt đến N gây tràn chỉ số.	Sửa biên: <code>range(len(lst))</code> .	✓
Vòng lặp vô tận (while)	Quên lệnh tăng biến đếm ở cuối thân vòng lặp.	Bổ sung lệnh: <code>i += 1</code> .	✓

Hãy duy trì tệp `Learning_Log.md` sau mỗi buổi học để theo dõi sự tiến bộ hàng tuần!

PHẦN 3

CHIẾN LƯỢC SỬ DỤNG AI (CHATGPT) HỖ TRỢ HỌC TẬP

AI Tutor, 4-Part Prompt, 5T Principle, Prompt Toolkit & Verification

12. Định vị Vai trò Chuẩn xác của AI trong Học Lập trình

CẢNH BÁO: Thói quen lạm dụng AI

Quy trình sai lầm:

Đề bài → Giao AI → Chép code → Bế tắc!

- ▶ Hoàn toàn không hiểu tại sao câu lệnh đó được dùng.
- ▶ Vào phòng thi không có AI → Hoàn toàn bế tắc.
- ▶ Kỹ năng tư duy logic bị thoái hóa theo thời gian.

KHUYẾN NGHỊ: AI là Huấn luyện viên (Coach)

Quy trình chuẩn mực:

Tự nghĩ → Thử code → Xin gợi ý → Tự làm!

- ▶ Nhờ AI **giải thích** khái niệm khó bằng ngôn ngữ đời thường.
- ▶ Nhờ AI **gợi ý từng bước** (không xin trọn vẹn đáp án).
- ▶ Nhờ AI **đặt câu hỏi ngược lại** để kiểm tra độ hiểu.
- ▶ Nhờ AI **review code** do chính mình tự viết.

13. Khung Cấu trúc Prompt Chuẩn mực 4 Thành phần

Thành phần	Ý nghĩa câu hỏi	Ví dụ minh họa thực tế
1. Context	Bạn là ai, cấp độ nào?	<i>"Tôi là sinh viên năm nhất đang bắt đầu học lập trình Python cơ bản..."</i>
2. Problem	Bạn đang vướng chỗ nào?	<i>"Tôi chưa phân biệt được sự khác nhau giữa biến số và giá trị gán..."</i>
3. Task	Muốn AI làm gì cụ thể?	<i>"Hãy giải thích bằng 1 ví dụ đời thường và 1 đoạn code Python dưới 8 dòng..."</i>
4. Constraint	Ràng buộc câu trả lời?	<i>"Không dùng kiến thức hàm/OOP, đặt cho tôi 2 câu hỏi kiểm tra, chưa vội đưa đáp án."</i>

Sự khác biệt:

So với câu hỏi cộc lốc *"Biến trong Python là gì?"*, prompt có cấu trúc giúp AI cá nhân hóa câu trả lời đúng 100% trình độ của bạn!

14. Nguyên tắc 5T khi Tương tác cùng AI



- ▶ **Think:** Xác định Input, Output, thuật toán trước khi mở khung chat.
- ▶ **Try:** Tự gõ bản phác thảo đầu tiên, dù có lỗi.
- ▶ **Talk:** Cung cấp đoạn code hiện có và hỏi đúng chỗ tắc.
- ▶ **Test:** Luôn chạy code trên IDE, thử các dữ liệu biên (số 0, số âm, mảng rỗng).
- ▶ **Teach-back:** Đóng khung chat lại và tự giải thích lại giải pháp bằng lời mình.

15. Hệ thống Gợi ý 3 Cấp độ (Three-Level Hint System)

Khi làm bài tập khó, hãy yêu cầu AI cấp gợi ý theo từng nấc – Không xin đáp án trọn vẹn!

Cấp 1: Gợi ý Ý tưởng

(Idea Hint)

Chỉ phân tích đề, chỉ ra hướng tiếp cận logic toán học/thuật toán.

→ **Tuyệt đối không có dòng code nào!**

Cấp 2: Gợi ý Cấu trúc

(Structure Hint)

Gợi ý câu lệnh nên dùng: vòng for, lồng if, toán tử chia lấy dư %...

→ **Định hình khung logic.**

Cấp 3: Khung sườn Code

(Scaffold Hint)

Đưa ra khung code có vị trí trống (# TODO) để tự suy nghĩ điền nốt.

→ **Giữ quyền làm chủ bài giải.**

Chiến lược làm bài: Hãy nỗ lực giải bài ở Gợi ý Cấp 1. Chỉ mở Cấp 2 và Cấp 3 khi thực sự bế tắc sau 15 phút suy nghĩ!

16. Prompt Toolkit Thực chiến (1): Khái niệm & Gọi ý

Học khái niệm mới (Concept Prompt)

Tôi là người mới bắt đầu học Python. Hãy giải thích cho tôi khái niệm [Hàm đệ quy].

Yêu cầu:

1. Bản chất dễ hiểu kèm 1 ví dụ đời thực.
2. Đoạn code mẫu ngắn (dưới 10 dòng).
3. 1 lỗi phổ biến người mới hay mắc.
4. Đặt 2 câu hỏi kiểm tra, chưa vội đưa đáp án.

Xin gợi ý bài tập (Socratic Hint)

Tôi đang tự làm bài: "[NỘI DUNG ĐỀ BÀI]"

Code tôi đã thử viết:

[DÁN ĐOẠN CODE CỦA BẠN]

Tôi đang bị tắc ở bước [MÔ TẢ VƯỚNG MẮC].

Yêu cầu: TUYỆT ĐỐI KHÔNG viết code giải hộ. Hãy cho tôi GỢI Ý CẤP ĐỘ 1 về mặt ý tưởng để tôi tự suy nghĩ tiếp.

17. Prompt Toolkit Thực chiến (2): Debug & Code Review

Hỗ trợ Debugging (Tìm nguyên nhân)

Tôi viết code Python sau đây: [MỤC TIÊU CỦA CODE]

Code hiện tại: [DÁN CODE]

Lỗi nhận được: [DÁN THÔNG BÁO LỖI]

Yêu cầu:

1. Giải thích nguyên nhân tại sao có lỗi này.
2. Chỉ rõ dòng code đang tiềm ẩn vấn đề.
3. Đặt cho tôi 1 câu hỏi gợi mở để tôi tự sửa, KHÔNG sửa lại toàn bộ code.

Review & Tối ưu mã nguồn (Code Review)

Tôi đã hoàn thành xong bài tập này và code chạy đúng: [DÁN CODE ĐÃ CHẠY ĐÚNG]

Hãy đóng vai trò Giảng viên Python:

1. Đánh giá đặt tên biến, tính rõ ràng, cấu trúc logic.
2. Code có xử lý được các trường hợp biên (edge cases) không?
3. Gợi ý 1 điểm có thể viết gọn hơn chuẩn Pythonic và giải thích lý do.

18. Prompt Toolkit Thực chiến (3): Tạo Quiz Ôn tập

Tạo Bộ đề Quiz tự kiểm tra kiến thức (Self-Quiz Prompt)

Tôi vừa học xong các chủ đề: **[List, Tuple, Slicing và Vòng lặp for]**.

Hãy tạo cho tôi một bài kiểm tra ngắn gồm 6 câu hỏi:

- ▶ **2 câu Trắc nghiệm:** 4 lựa chọn A, B, C, D (chỉ 1 đáp án đúng).
- ▶ **2 câu Dự đoán kết quả (Predict the Output):** Đoạn code 3–5 dòng không dùng hàm nâng cao.
- ▶ **2 câu Tìm lỗi sai (Find the Bug):** Đoạn code có đúng 1 lỗi cần phát hiện và sửa.

Ràng buộc: KHÔNG hiển thị đáp án trước. Hãy chờ tôi gửi câu trả lời rồi mới chấm điểm và giải thích cặn kẽ từng câu!

Mẹo nâng cao: Quiz từ chính lỗi sai trong quá khứ

Sao chép các dòng lỗi trong `learning_log.md` của bạn vào prompt và yêu cầu: *"Hãy tạo 5 câu hỏi xoay quanh đúng những lỗi sai này để kiểm tra xem tôi đã thực sự khắc phục được chưa!"*

19. Bảng kiểm 7 Tiêu chí Thẩm định Thông tin từ AI

- ☐ **1. Tính tương thích:** Code AI đưa ra có dùng thư viện lạ hay kiến thức vượt phạm vi môn học không?
- ☐ **2. Chạy thử nghiệm:** Đã copy code vào IDE và chạy thử không có lỗi biên dịch/runtime chưa?
- ☐ **3. Khớp đề bài:** Kết quả in ra có đúng từng chữ so với yêu cầu đầu ra của bài toán không?
- ☐ **4. Kiểm tra biên:** Code có chạy đúng khi input là số 0, số âm, danh sách rỗng?
- ☐ **5. Thuật ngữ chuẩn:** Thuật ngữ AI giải thích có đồng nhất với giáo trình chính thức môn học không?
- ☐ **6. Tự giải thích từng dòng:** Bạn có thể chỉ tay vào từng dòng code và giải thích tại sao nó ở đó?
- ☐ **7. Tái tạo độc lập:** Nếu tắt AI đi, bạn có thể tự mình viết lại code này trong 10 phút không?

Quy tắc Vàng Bất di Bất dịch:
"TUYỆT ĐỐI KHÔNG BAO GIỜ NỢP HOẶC LƯU LẠI MÃ NGUỒN
MÀ BẢN THÂN BẠN KHÔNG THỂ TỰ MÌNH GIẢI THÍCH TƯỜNG TẬN!"

20. Kế hoạch Hành động Ngay trong Tuần Đầu tiên

1. Chuẩn bị Môi trường

- ▶ Cài đặt **Python 3.11+** từ trang chủ `python.org`.
- ▶ Cài đặt trình soạn thảo **VS Code** kèm extension Python.
- ▶ Đăng nhập sẵn tài khoản **Google Colab** trên trình duyệt.

2. Khám phá Học liệu

- ▶ Bookmark kho GitHub học phần: `github.com/vudmvn/Python`.
- ▶ Đọc kỹ Đề cương chi tiết (`syllabus-vn.md`).
- ▶ Đọc trước Chương 1 giáo trình *Python for Everyone*.

3. Sổ tay Tự học

- ▶ Tạo thư mục học tập `DSAI1003_Python/` trên máy.
- ▶ Tạo tệp `learning_log.md` ghi chép sau mỗi tuần.
- ▶ Lưu sẵn bộ khung Prompt Toolkit để tương tác AI văn minh.

**CHÚC CÁC BẠN SINH VIÊN KHOA KH DỮ LIỆU & TRÍ TUỆ NHÂN TẠO
LÀM CHỦ TƯ DUY LẬP TRÌNH & BỨC PHÁ NĂNG LỰC CÙNG KỶ NGUYÊN AI!**

THANK YOU!

CẢM ƠN CÁC BẠN ĐÃ LẮNG NGHE

TS. Vũ Đức Minh

Khoa Khoa học Dữ liệu và Trí tuệ Nhân tạo – Trường Công nghệ
Đại học Kinh tế Quốc dân (NEU)

Website:

<https://vudmn.github.io/Fundamental-of-Programming-Concepts-in-Python/>